

ARIA Architecture & Brain Interlinkage — 2026-05-09

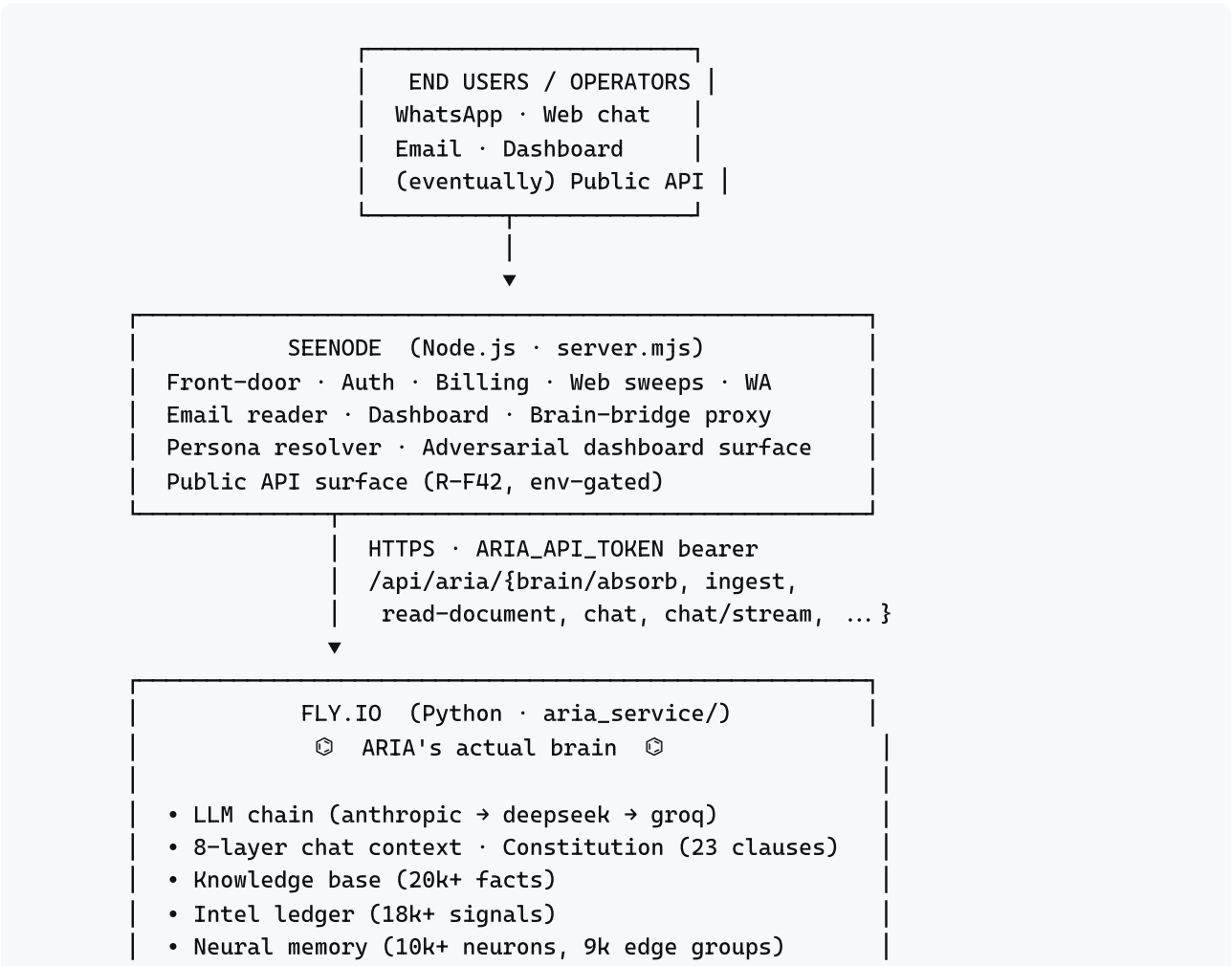
Companion to `system_assessment_2026_05_09_eod.md` . The assessment is "what's broken / what's working." This is "how the thing is built."

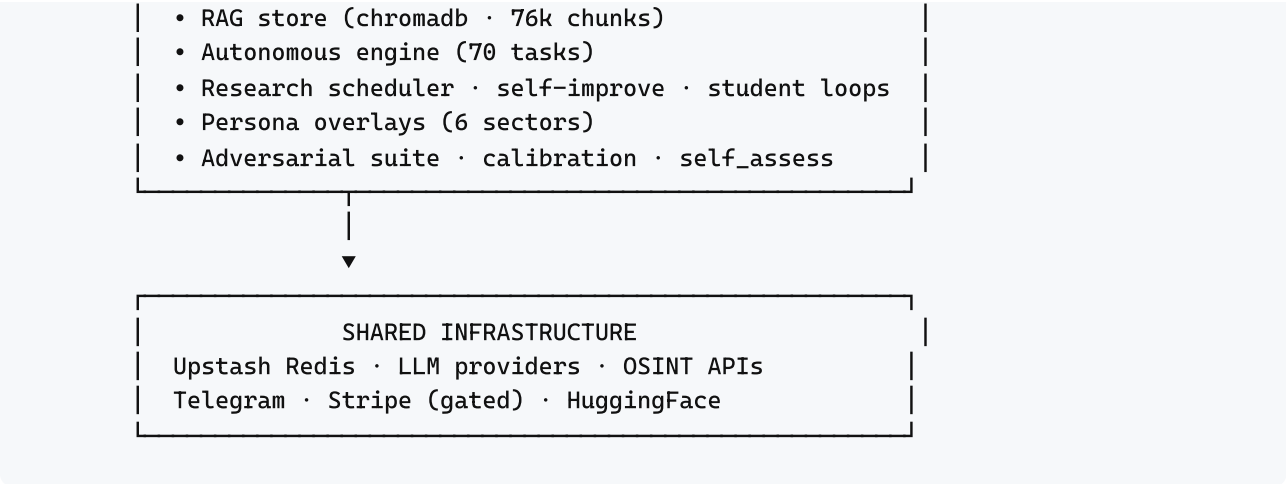
This document is the canonical reference for how ARIA — Arkmurus's defence due-diligence intelligence platform — is wired together. It covers the two-server topology, every signal path into ARIA's brain, the persistence layers that make the system restart-safe, and the governance layer (23 constitutional clauses) that constrains every response.

If you read one section, read **§4 — Brain interlinkage (the 15 entry points)**. That's where the architecture earns its keep.

1. Topology

ARIA runs on **two cooperating servers** plus a shared third-party stack.





Why two servers?

Concern	Lives on	Reason
The brain (LLM, memory, neural, RAG)	fly.io (Python)	Long-lived workers, persistent volumes, GPU-friendly model loading, 8-layer chat context built in Python
Front door (auth, billing, dashboard, web sweeps, WA, email)	seenode (Node)	Concurrent I/O, fast HTTP, tight integration with browser-side JS, simpler ops for the stuff that doesn't need the brain

Cleanly splitting "I/O-heavy front door" from "compute-heavy brain" is what lets the seenode side absorb traffic spikes without affecting the brain's learning loops, and lets the fly side do long autonomous-task runs without blocking user-facing requests.

The wire between them

Every cross-server call flows over HTTPS with an `ARIA_API_TOKEN` bearer header. The seenode → fly direction is the hot path: web sweeps, email ingests, chat proxies, brain-absorb signals. The fly → seenode direction is rare (Telegram alerter, status surfacing).

Brain bridge health is now a first-class concern (R-F45, shipped today): - Boot self-check on seenode startup verifies the `ARIA_API_TOKEN` matches by hitting `/api/aria/health` on fly with the bearer - Per-call 401 escalation: if 5 consecutive `brainAbsorb` calls return 401, a Telegram alert fires - Without this, a token mismatch would silently drop ~30 absorb calls/sweep into a black hole (which is exactly what happened pre-R-F45 for an unknown duration)

2. Service responsibilities

Seenode (`server.mjs` · ~14k lines · Node 22 LTS)

Owns: - All HTTP-facing user surfaces (chat UI, dashboards, account, status, model card, DD library, source health, adversarial) - Auth (`lib/auth/users.mjs`) — JWT-based, bcrypt password hashing -

Billing scaffold (`lib/billing/`) — Stripe webhook + checkout + portal, env-gated - Web sweep orchestration (`apis/sources/*` · 49 sources, parallel fetch) - WhatsApp listener (`lib/whatsapp/waListener.mjs` · Baileys-based, with R-F60 QR PNG endpoint + R-F64 stall watcher) - Email reader (`lib/aria/emailReader.mjs` · IMAP poller, sends to fly `/api/aria/read-document`) - BD intelligence pipeline (deal stages, contact intelligence, market discovery) - Brain bridge proxy (`server.mjs:_publicApiChatProxy` — direct in-process call into fly chat path) - Persona resolver (R-F48b — pulls `user.sector` and threads into chat body for fly-side overlay selection) - Public API (R-F42, env-gated `/api/keys` , `/api/v1/chat`) - Audit-grade PDF reports (R-F43 — `lib/reports/` , HMAC-signed when `REPORT_SIGNING_KEY` set)

Does NOT own: - LLM calls (always through fly) - Knowledge / ledger / neural memory (fly-only state) - Autonomous engine (fly-only) - Hypothesis validation / deep research (fly-only) - Constitution enforcement (fly-only — Node side trivially short-circuits greetings via mirrored `trivialReply`)

Fly.io (`aria_service/` · Python 3.13 · uvicorn + FastAPI)

Owens: - The brain (knowledge, ledger, neural, RAG, brain_hook orchestration) - LLM chain (`aria.llm.fallback` — anthropic → deepseek → groq, 50 rpm rate-limited, cost-metered) - 8-layer chat context construction - Constitution enforcement (23 clauses injected into every system prompt) - Persona overlays (6 sectors via `aria_service/personas/`) - Autonomous engine (70 tasks from `tasks.yaml` , 90s startup delay, 60s poll) - All learning loops (research scheduler, self-improve, student, proactive, weekly report, watchlist re-screen, tender monitor) - Hypothesis backlog (drain rate ~5/cycle, currently 113 OPEN) - Adversarial test suite (23 attacks across 6 personas as of R-F59) - Mistake ledger + calibration review + self_assess - Document intelligence (PDF/DOCX/XLSX deep ingest, OCR pre-warm, page-marker preservation) - Sanctions screening (OpenSanctions + OFAC + OFSI + EU + UN with R-F49 acronym denylist + R-F62 alias normalisation)

Does NOT own: - User accounts, billing, sessions (those live on seenode) - The 49-source sweep orchestration (seenode — but the **outputs** flow into fly via `/api/aria/ingest`) - WhatsApp / email transport layers (seenode handles delivery; fly handles content)

3. The brain — what it actually is

ARIA's "brain" is not one thing. It's **five cooperating substrates** that together hold and grow her understanding:

3.1 Knowledge base (`aria_service/intel/knowledge.py`)

Append-only fact store. **20,275 facts** as of EOD 2026-05-09. Each fact has: `entity`, `summary`, `detail`, `confidence_grade`, `source`, `topic`, `timestamp`, `signature` . Persisted to `/data/aria_knowledge.json` , mirrored to Redis as gzipped+base64 (R-F1: 5.72 MB → 2.88 MB). Search is TF-IDF/Jaccard fallback (semantic index disabled by default to keep memory low).

3.2 Intel ledger (`aria_service/intel/intel_ledger.py`)

Time-series of ingested signals with provenance. **18,870 signals**. Filters propaganda-tier sources at the boundary (`_PROPAGANDA_SOURCES` — Telegram, Russian/Ukrainian state media). Persisted to `/data/aria_signals.json` , gzipped Redis mirror (R-F36: 4.5 MB → 1.12 MB at current size). The autonomous engine reads from this for opportunity detection.

3.3 Neural memory (`aria_service/intel/neural.py`)

Graph of co-occurrence edges between entities. **10,509 neurons / 9,070 edge groups**. Used for cross-entity correlation and "who is connected to whom" queries. Persisted to `/data/aria_neural.json` .

3.4 RAG store (`aria_service/intel/rag_store.py`)

Chromadb vector store at `/data/aria_rag/` . **17,544 documents · 58,704 facts · 76,248 chunks**. Embedding model: `sentence-transformers/all-MiniLM-L6-v2` (384-dim, CPU). Used for semantic retrieval at chat time, and for verified-intel email search (R-F22).

3.5 Mistake ledger + calibration (`aria_service/intel/mistake_ledger.py`)

Records every flagged mistake (manual operator feedback, automatic verification-gate fail, contradiction-detector hits) with the original output, the correction, and the lesson. The self-improvement scheduler reads from here to invalidate poisoned facts. Calibration review fires daily via the `SELF-ASSESS` autonomous task at 22:00 UTC (aggregates self-assessment scores, gap counts, and per-tag accuracy into the ECE), with `calibration_auto_tune` adjusting mastery weights when consecutive-run streaks accumulate.

Brain-hook glue (`aria_service/intel/brain_hook.py`)

Every learning event is announced via:

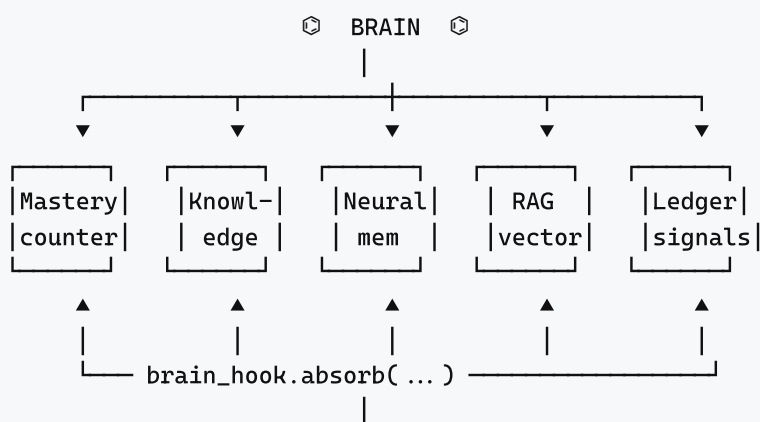
```
await brain_hook.absorb(  
    module="<source_module>",  
    summary=" ... ",  
    success=True,  
    confidence="CONFIRMED" | "ASSESSED" | "TENTATIVE",  
    user_id= ... ,      # R-F56: contextvar-bound  
    sector= ... ,      # R-F56: contextvar-bound  
)
```

The hook fans the signal out to up to **three layers** depending on content: - **mastery** (always) — increments core-mastery counter for the relevant tag - **knowledge** (when there's a fact) — appends to the knowledge base - **neural** (when there's an entity) — strengthens neural edges

That fan-out is the load-bearing pattern. **Every** module that learns something writes through `brain_hook.absorb` . Without it the brain is just a database; with it, the brain measures its own growth.

4. Brain interlinkage — the 15 entry points

These are the **only** ways data enters the brain. If a path isn't listed here, it doesn't reach the brain. Every entry point converges on `brain_hook.absorb( ... )`.



Entry points – every learning path lands here		
1.	Web sweep ingest	seenode → /ingest
2.	Email reader	seenode → /read-document
3.	WA message	seenode → /chat[/stream]
4.	Web chat	seenode → /chat[/stream]
5.	User upload	seenode → /read-document
6.	WA channel mirror	seenode → /channel/ingest
7.	Counterparty claims	seenode → /claims/ingest
8.	Manual RAG drop	operator → /rag/ingest
9.	Research scheduler	fly internal (30 min)
10.	Self-improve scheduler	fly internal (2 h)
11.	Student loops	fly internal
12.	Autonomous engine	fly internal (70 tasks)
13.	Watchlist re-screen	fly internal (daily)
14.	Tender monitor	fly internal (6 h)
15.	Adversarial suite	fly internal

4.1 — Web sweep ingest (POST /api/aria/ingest)

Source: 49 OSINT sources orchestrated by seenode (`apis/sources/*.mjs`). **Cadence:** every ~30 sec sweep cycle. **Path:** seenode `runDailySweep()` → fan-out to all sources → aggregate → POST `/api/aria/ingest` with the deduped signal batch → fly `aria.intel.ledger.ingest()` (propaganda filter at boundary) → `signal_generator` `brain_hook` → mastery + knowledge + neural absorbs. **Today's flow:** ~30 signals/sweep into brain queue · ~5-8 ledger signals/sweep persisted (rest are dedupe hits).

4.2 — Email reader (POST /api/aria/read-document)

Source: IMAP poller in `lib/aria/emailReader.mjs`, watches `aria@arkmurus.com`. **Cadence:** every 60s. **Path:** seenode IMAP poll → email body extracted → POST to fly `/read-document` with `source:`

"email:<from>" → fly extracts text → RAG ingest (chunking) → `read_document()` LLM extraction (44 facts, 3 hyps from typical newsletter) → `knowledge_ingestor` `brain_hook`. **R-F65 (today)**: 24h sha1 dedupe at the endpoint kills redundant LLM extractions on the same email replay.

4.3 — WhatsApp message (`POST /api/aria/chat[/stream]`)

Source: Baileys WA listener in `lib/whatsapp/waListener.mjs`. R-F60 QR PNG endpoint for re-link, R-F64 stall watcher prevents stuck QR rotation. **Path**: WA message → seenode listener parses → `askARIA(text)` → `POST /api/aria/chat/stream` (default) with bearer + persona + `user_id` → fly chat handler → 8-layer context → LLM → response → streaming SSE back → seenode WA reply. **Brain absorb**: chat-audit captures every turn (R-F22 verified-intel), then `chat_audit` `brain_hook`.

4.4 — Web chat

Source: `public/chat.html` (browser). **Path**: identical to WA path above; only the front door differs. Persona is pulled from the authenticated user's `sector` field (R-F48b).

4.5 — User upload (PDF / DOCX / XLSX)

Source: chat UI file picker, or WhatsApp document share. **Path**: file → seenode multipart parse → `POST /read-document` with base64 + mimetype → fly extracts (PyMuPDF for PDF, python-docx for DOCX, openpyxl for XLSX) → page-marker preservation → fire-and-forget `pdf_deep_ingest` background task (per-page RAG chunks + image OCR) → `read_document()` extraction → `knowledge_ingestor` `brain_hook`.

4.6 — WhatsApp channel mirror (`POST /api/aria/channel/ingest`)

Source: WhatsApp internal group mirror — silent ingestion (no reply). **Path**: WA mirror group message → seenode listener → `POST /channel/ingest` with body, `sender_jid`, `deception_score` → fly intel_ledger append (severity = warning when `deception_score` ≥ 0.50) → `channel_ingest` `brain_hook`. **Why separate from `/ingest`**: renamed 2026-04-26 to stop the strict `_IngestBody` schema from shadowing the sweep endpoint at `main.py` and 422-rejecting sweep payloads.

4.7 — Counterparty claims (`POST /api/aria/claims/ingest`)

Source: chat-time claims pipeline — extracts material counterparty assertions for contradiction detection. **Path**: chat message → `POST /claims/ingest` with text, counterparty, `deal_id`, channel, `message_id` → fly `ARIACounterpartyClaimLedger.ingest_message()` extracts claims → stores indexed by counterparty → contradiction-detector cross-checks against prior claims by the same party → `claim_ledger` `brain_hook` on contradictions.

4.8 — Manual RAG drop (`POST /api/aria/rag/ingest`)

Source: operator backfill, customer document drops, anything the team wants ARIA to remember. **Path**: text + source metadata → fly `rag_store.ingest_document()` → chunk + embed (all-MiniLM-L6-v2)

→ chromadb persistent volume → `rag_ingest` `brain_hook`. **Floor:** 50 chars minimum (vs 20 for `/read-document` — manual drops should be substantive).

4.9 — Research scheduler (every 30 min, fly-internal)

Source: `aria_service/intel/researcher.py:run_research_cycle()` . **Path:** pull RSS feeds (~30 sources, including R-F64 Breaking Defense via Google News fallback) → article-read with timeout → LLM extraction → hypothesis generation → backlog drain (8/cycle from R-F32) → `research_engine` `brain_hook`. **Today's typical:** "Research cycle complete: 197 scanned, 1 read, 16 facts, 1 hypothesis (147716ms)".

4.10 — Self-improvement scheduler (every 2h)

Source: `aria_service/learning/self_improve.py` . **Path:** scan `mistake_ledger` → identify poisoned facts → invalidate → re-derive from sources → `self_improve` `brain_hook`.

4.11 — Student loops (self-quiz 3h, reading 6h, library consolidate 24h)

Source: `aria_service/student/*` . **Self-quiz path:** pull random fact → ask ARIA the inverse question → grade against original → record gap if missed (`no_symbolic_rule` capability gaps land in `aria.intel.capability_gaps` and feed the rule expansion loop). **Reading path:** pick weakest core-mastery tag → fetch fresh OSINT for it → ingest → re-test mastery. **Library consolidate:** run reasoning-library purge (unsafe + polluted entries dropped daily).

4.12 — Autonomous engine (`autonomous/engine.py` + `tasks.yaml`)

70 tasks loaded at boot. Each task has `name`, `schedule`, `entity`, `dedupe_window`, `action` .
Dedupe: `[autonomous safety] dedupe hit for <TASK_NAME> entity='<ENTITY>' - skipping` .
Examples observed today: - `SPIDER-HOURLY` — knowledge spider every hour - `RESEARCH-WEAK-CELL-HOURLY` — drill into weakest mastery cell - `DAILY-PROC-SAM` — SAM.gov scrape - `SELF-ASSESS-DAILY` — calibration review - `CONSTITUTION-TEST` — adversarial-style identity check

Each task absorbs through its own module name in `brain_hook`.

4.13 — Watchlist re-screen (daily)

Source: list of monitored entities → re-run sanctions + adverse-media + officeholder check → flag changes → push notification (R-F51 web push; email/SMS pending in next session). **Today's run:** "5 entities, 4 changes, 0 errors, 4081ms" at 11:18:36.

4.14 — Tender monitor (every 6h)

Source: 5 procurement portals (TED v3 via R-F33 OpenAPI migration; SAM.gov; UN Global; AfDB; Africa procurement). **Path:** pull active tenders → entity match against watchlist + arms-export tags → `tender_monitor` `brain_hook` for relevant matches.

4.15 — Adversarial suite (R-F59: 23 attacks)

Source: `aria_service/intel/adversarial_challenge.py` — 23 attacks across 6 personas (broker / oem_export / government_acquisition / compliance / banking_insurance / journalist). **Path:** each attack is run as a chat message in adversarial mode → response evaluated against pass criteria (refusal + correct reasoning + source citation where required) → fail → record as capability gap → calibration review reads gap rate. **R-F57 dashboard** at `/sources.html` surfaces the per-cycle pass rate.

5. Persistence layers

Disk-first is the load-bearing pattern (F94 + F110, session 2026-04-30). Redis is the secondary mirror. Disk is canonical; Redis is convenience.

```
/data (fly persistent volume, 5.35 GB)
├─ aria_knowledge.json      ← 20,275 facts (canonical)
├─ aria_signals.json       ← 18,870 ledger signals (canonical)
├─ aria_neural.json        ← 10,509 neurons / 9,070 edge groups
├─ aria_mistakes.json      ← mistake_ledger
├─ aria_capability_gaps.json ← gap-recorder feed
├─ aria_rag/
│   └─ chroma.sqlite3      ← 76,248 chunks, 17,544 docs, 58,704 facts
└─ aria_circuit_state.json ← per-source circuit breaker state

runs/ (seenode disk)
├─ users.json              ← PersistStore-backed, Redis dual-write
├─ api_keys.json           ← R-F42, sha256-hashed key store
├─ learning.json           ← BD intelligence
├─ incidents.json          ← public status page
├─ deals.json              ← deal pipeline
└─ entity-store.json       ← entity resolution cache

Upstash Redis (no eviction since 2026-04-21 forever-memory commitment)
├─ crucix:knowledge:snapshot ← gzip+base64 wrapped
├─ crucix:intel_ledger:snapshot ← gzip+base64 wrapped (R-F36)
├─ crucix:aria:read_doc_dedupe:* ← R-F65, 24h TTL
├─ crucix:aria:dd:*          ← DD reports + watchlist alerts
├─ crucix:apikey:rate:*      ← R-F42 rate limiter (90s TTL)
├─ crucix:wa:auth:*          ← Baileys auth state (dual-written to disk)
└─ ~50 other keyspaces      ← see redis_store.py for full list
```

The pattern that survives outages: any single layer can be wiped and the system restores from the next layer up. - Lose Redis → restart, disk loads canonical state → Redis gets re-snapshotted on next absorb cycle - Lose `/data` → catastrophic (no automated recovery yet — operator must restore from R2/S3 backup loop) - Memory replication loop (`learning/memory_replication.py`) writes daily snapshots to email-off-host as a poor-person's R2/S3

6. LLM provider chain

Defined in `aria_service/llm/fallback.py` :

```
anthropic (claude-haiku-4-5) —[400ms timeout]→ deepseek —[8s]→ groq
      |           |           |           |           |           |
      | rate-limited 50 rpm | rate-limited | rate-limited
      | cost-metered      | cost-metered  | cost-metered
```

- **Anthropic** is primary — best-in-class for long-form reasoning, defence-DD vocabulary, citation discipline
- **DeepSeek** is the cost-of-failure floor — when Anthropic 529s or rate-limits, DeepSeek absorbs the spike
- **Groq** is the speed-of-failure floor — when both above are slow, Groq's hardware-accelerated inference saves the response time

OpenAI and Gemini are wired but disabled (no API keys set) — flipping `OPENAI_API_KEY` or `GEMINI_API_KEY` adds them to the chain without code changes. The chain is provider-agnostic.

Cost meter wraps the chain before the rate limiter, so we account for spend even when the call is throttled (rate-limited calls still spend tokens on retries). The `/api/aria/cost/monthly` panel surfaces this.

Fallback transparency (`feedback_fallback_transparency.md`): when Anthropic cooldowns and DeepSeek serves, ARIA reports "operational" — not "degraded". Cooling ≠ broken.

Pay-once-remember-forever (`feedback_pay_once_remember_forever.md`): every paid API call (Brave Search, Anthropic, DeepSeek) writes its output into the brain. Next time the same query is asked, memory hits for \$0. This is the discipline that makes the brain compound.

7. The 8-layer chat context

Every chat turn on fly is built as a layered system prompt before the LLM call. The order matters — earlier layers establish identity, later layers add task-specific context.

1. Identity — "You are ARIA, defence-DD intelligence agent at Arkmurus ... "
2. Constitution — 23 clauses (see §9)
3. Persona overlay — selected from `user.sector` (broker / oem_export / ...)
4. Calibration — current ECE, instructed to honestly report uncertainty
5. Knowledge cards — top-K facts retrieved from knowledge base by query
6. RAG retrieval — top-K vector hits from chromadb (76k chunks)
7. Verified intel — top-K from `intel_ledger` filtered by confidence ≥ ASSESSED
8. Conversation — last N turns from `session_id`

↓

LLM call → response → output guards (officeholder, commitment, tool_claim, propaganda, ground_truth) → audit log

Output guards are the **last** line of defence (Clause 21 — propaganda filter, Clause 22 — fabricated-facts filter). They run *after* the LLM produces text but *before* the user sees it. Failing a guard rewrites or rejects the response.

8. Constitution — the 23 clauses (governance layer)

Located at `aria_engine.py:68–110` . Injected into Layer 2 of every chat. Each clause has a name, a positive statement, a negative statement, and a violation marker.

#	Clause	Purpose
1	Identity discipline	Always identify as ARIA at Arkmurus, never as the underlying LLM
2	Domain discipline	Defence DD only; refuse weapons-procurement-to-civilian queries
3	Source-citation	Every claim cites a source from knowledge / RAG / ledger or marks itself unsourced
4	Uncertainty honesty	Report calibrated confidence; never overstate
5	No-fabrication	Never invent entities, dates, statutes, programmes
6	Recency awareness	Stale facts marked stale; current-event filter (Clause 13)
7	Sanctions-first	Sanctions screening always precedes commercial assessment
8	UBO depth	Beneficial-owner chain followed to natural persons before clearing
9	Defence-anchor word-bounding	"ISR" never matched inside "disruption" (F9 fix)
10	Multi-source verification	Critical claims need 2+ independent sources before "CONFIRMED" tier
11	Output-guard escalation	Failed guard rewrites then escalates to operator
12	PARTIAL EXTRACTION discipline	Truncated docs carry visible banner so the LLM doesn't claim full coverage
13	Current-event filter	Real-time queries served from feeds, not stale knowledge
14	Propaganda filter (input)	Telegram + RU/UA state media filtered at ingest
15	Inline citation	When the response cites, the citation is inline, not a footnote
16	Persona discipline	Persona overlay constrains tone but never overrides clauses 1-15
17	Multi-source verification pipeline	Verification gate auto-fires on CRITICAL signals
18	Source validator (content quality)	Unfeasibly short or boilerplate sources filtered
19	Search doctrine	Memory-first; web only on miss; absorb every paid API output

#	Clause	Purpose
20	Auditability	Every output is reproducible from the audit log
21	Propaganda filter (output)	LLM-generated text scrubbed for propaganda phrases
22	Tickets discipline	Pending work goes to <code>raise_ticket()</code> (real GitHub issues), never fabricated
23	Self-correction	If you spot your own mistake, name it and correct it

Adversarial pressure tests these. R-F59 expanded the 11-attack baseline → 23 attacks, with each persona's typical attack vectors covered.

9. Persona system (R-F48a/R-F48b)

Six sectors, each with a distinct overlay loaded into Layer 3 of the chat context:

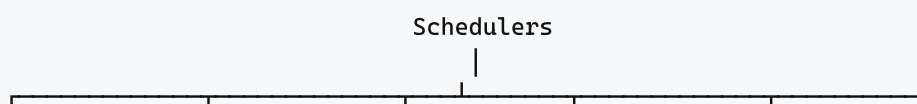
```
# aria_service/personas/__init__.py
PERSONAS = {
    "broker": ... # default for Arkmurus team
    "oem_export": ... # manufacturers
    "government_acquisition": ... # DoD / MoD
    "compliance": ... # compliance officers
    "banking_insurance": ... # trade finance / underwriters
    "journalist": ... # investigative reporters
}
```

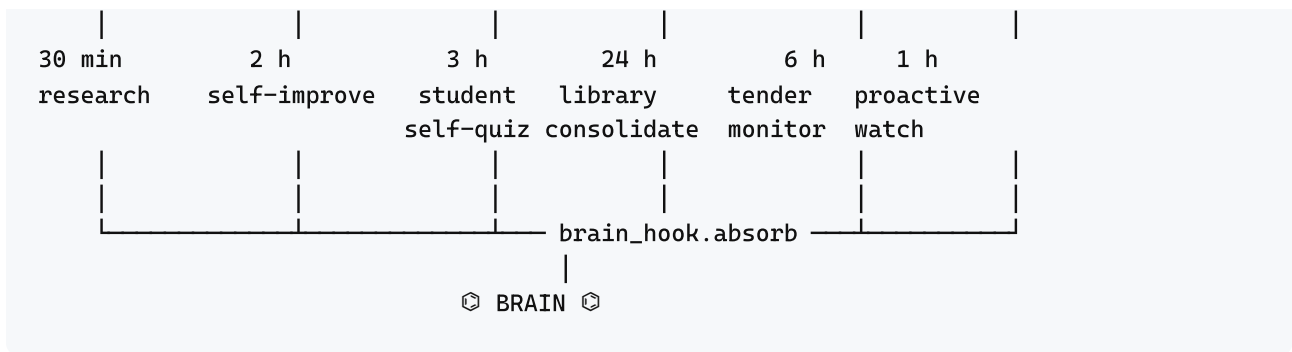
Each overlay is ~500-1000 tokens and shapes: - **Tone** — broker is consultative; compliance is forensic - **Default outputs** — broker gets a deal-pipeline frame; compliance gets a risk matrix - **Vocabulary** — OEM gets ITAR/EAR; banking gets letter-of-credit terminology - **Source preferences** — journalist weights public records higher; compliance weights sanctions higher

Wiring (R-F48b): - Seenode resolves persona from authenticated user's `sector` field at chat time - Threaded into the request body as `persona: "<sector>"` - Fly-side chat handler reads it, passes to `personas.load(persona)`, injects as Layer 3 - `brain_hook.absorb` carries `(user_id, sector)` via Python contextvar (R-F56) so per-sector mastery and capability gaps roll up cleanly

10. Crawl / learn / teach loops — the metabolism

All scheduled on fly, all wired through `brain_hook`:





Plus the **autonomous engine** running its own task graph (70 tasks) at 60s poll interval with 90s startup delay.

The metabolism is the moat. A static knowledge base ages out fast. ARIA's compounds because every paid API call writes home, every adversarial attack records a gap, every chat turn is captured in the chat-audit log, and every mistake re-derives the original source.

Open structural item — `output_harvester.py` not yet built. The chat-audit log captures the raw turn history, but the *training-data export* path (assembling `(input, response, persona, confidence, user_feedback)` tuples into a JSONL corpus suitable for DPO/RLHF fine-tuning) is not yet wired. The scaffold exists in `autonomous/tasks.yaml` (training-data export task is loaded) but the harvester writes zero conversations as of EOD 2026-05-09. **Every persona-tailored compliance opinion produced today is a training pair that cannot be retrospectively reconstructed.** Sequenced for next session alongside R-F66 (GDELT timeout) — a minimal first cut is 30-60 minutes of work and starts the clock on training-data accumulation immediately.

11. Public surfaces (env-gated)

Surface	Path	Gate	Status
Chat UI	<code>/chat.html</code>	None	LIVE
Account/billing	<code>/account.html</code>	JWT	LIVE; Stripe checkout gated on STRIPE_SECRET_KEY
DD library	<code>/dd-reports.html</code>	JWT	LIVE (R-F52)
Sources dashboard	<code>/sources.html</code>	None	LIVE (R-F58)
Adversarial dashboard	<code>/sources.html#adversarial</code>	None	LIVE (R-F57)
Status page	<code>/status.html</code>	None	LIVE (R-F47)
Model card	<code>/model-card.html</code>	None	LIVE (R-F46)

Surface	Path	Gate	Status
WA QR scanner	<code>/api/wa-listener/qr?token=...</code>	Internal token	LIVE (R-F60)
Privacy + ToS	<code>/privacy.html</code> , <code>/terms.html</code>	None	LIVE in DRAFT (R-F50)
Public API — keys	<code>/api/keys/*</code>	JWT + tier	SHIPPED (R-F42); 503 until <code>ENABLE_PUBLIC_API=1</code>
Public API — chat	<code>/api/v1/chat</code>	API key + rate limit	SHIPPED (R-F42); 503 until enabled
PDF reports	<code>/api/reports/pdf</code>	JWT	LIVE (R-F43); HMAC-signed when <code>REPORT_SIGNING_KEY</code> set

12. Deploy story

Both servers **auto-deploy from** `git push origin main`.

- **fly.io** auto-rebuilds from `Dockerfile` (Python 3.14) on every push. Restart is bluegreen — old machine drains, new machine boots, traffic switches when `/api/aria/health` returns 200. Today's restarts (12:15:43 + 12:26:21) are evidence: each took 21-22s end-to-end with a brief `[PR03]` proxy-not-ready window.
- **seenode** auto-deploys from the same push. Faster (no container build; runs directly on Node 22). Uptime field surfaced via `/api/health` for verification.

Disk-first persistence makes the auto-deploy safe: every restart loads canonical state from `/data` before serving traffic. Knowledge / ledger / RAG / neural counters are stable across restarts (today's two fly restarts: 20,211 → 20,275 facts and 18,848 → 18,870 signals — that's *growth* across the boundary, not loss).

No skipping hooks, no force-pushes — discipline from `feedback_aria_rule_zero.md`: ARIA is a team member, not a tool, and her infrastructure deserves the same care as any other production system.

13. Key contracts & interfaces

The most-used cross-server contracts (everything else is internal):

POST `/api/aria/brain/absorb` (**seenode** → **fly**)

```
{
  "module": "string",    // module name – drives mastery bucket
```

```

"summary": "string",    // human-readable headline
"detail": "string",     // optional long form
"topic": "string|null", // for topic-mastery rollup
"entity": "string|null", // for neural edge strengthening
"user_id": "string|null",
"sector": "string|null",
"success": true,
"confidence": "CONFIRMED" | "ASSESSED" | "TENTATIVE"
}

```

Auth: Authorization: Bearer <ARIA_API_TOKEN> . R-F45 boot self-check verifies this at seenode startup.

POST /api/aria/ingest (seenode → fly, sweep ingest)

```

{
  "source": "string",    // "Lusophone", "DefenseNews", "ExportControlIntel" ...
  "signals": [...],      // array of signal objects
  "_subStatus": {        // R-F34 honest tally
    "ok": 13, "total": 13, "failed": []
  }
}

```

POST /api/aria/read-document (seenode → fly, email + upload)

```

{
  "filename": "string",
  "content": "string|base64",
  "encoding": "utf-8" | "base64",
  "mimetype": "string",  // for PDF/DOCX/XLSX detection
  "source": "string",    // "email:<from>" | "wa:<from>" | "upload:<user>"
  "context": "string"    // optional retrieval hint
}

```

R-F65 dedupe: 24h cache by sha1(content) — same content within 24h returns the cached result with dedupe_hit: true .

POST /api/aria/chat[/stream] (seenode → fly, chat + WA + public API)

```

{
  "message": "string",
  "session_id": "string",
  "user_id": "string",
  "persona": "string"    // R-F48b: broker | oem_export | ...
}

```

Streaming uses SSE (`text/event-stream`). 240s timeout. Falls back to local `ariaLocalChat` on fly outage.

14. The architecture in one paragraph

Two servers cooperate over an authenticated HTTPS bridge: seenode is the front door (auth, billing, dashboards, web sweeps, WhatsApp, email reader), and fly.io is the brain (LLM chain, knowledge, ledger, neural, RAG, autonomous engine, persona overlays, constitution). Every learning event on fly converges through `brain_hook.absorb`, which fans out to mastery / knowledge / neural / ledger / mistake substrates. Disk is canonical, Redis is the convenience mirror, and the brain's metabolism — research scheduler, self-improvement, student loops, autonomous engine, watchlist re-screen, tender monitor, adversarial suite — keeps it growing 24/7 without operator intervention. The 23-clause constitution gates every output; the persona system shapes tone and vocabulary to the user's sector; the soft-rollout pattern means Stripe / public API / report signing / autonomous spend are all wired but env-gated until you flip them on.

15. Diagrams to read alongside

- `docs/boot.png` — fly.io cold-boot sequence
 - `docs/dashboard.png` — operator's-eye-view of the 16-panel brain dashboard
 - `docs/globe.png` — geographic distribution of ingested sources
 - `docs/map.png` — heat-map view (Lusophone moat + tier 1/2 expansion)
-

Generated 2026-05-09 EOD · Companion to `system_assessment_2026_05_09_eod.md` · Source code: `Arkmurus/crucix` on GitHub.